

Detecting weak passwords in Active Directory – Michael Waterman



michaelwaterman.nl/2025/04/10/detecting-weak-passwords-in-active-directory

Michael Waterman

April 10, 2025

A little while ago, I wrote about the [shift toward passwordless authentication](#) and why we need to rethink our dependency on traditional passwords. While I'm still convinced that passwordless is the future, and the sooner we get there, the better. At the same time I also see the reality most environments are still dealing with.

During the audits I perform, I see organizations *massively struggling* with password policies and the ability to control weak passwords. That's yet another reason to move toward passwordless... but let's be honest:

| The truth is: we're not quite there yet.

Legacy systems, user behavior, and years of accumulated technical debt make a full transition challenging for many organizations. In the meantime, we can't afford to stand still. We have to proactively reduce the risk that passwords still pose.

One of the most overlooked, but incredibly effective, ways to do that is by checking whether your Active Directory passwords are already publicly available in known data breaches. Because yes, your users might be reusing credentials that are floating around in lists like [Have I Been Pwned](#).

Now, you might be thinking: **“But Mike, isn't this old news?”**

And you'd be absolutely right. The concept isn't new, but I've found that many people still have no idea how to *actually do this*, and often fall back on commercial tools. While there's nothing wrong with that, it does raise questions around:

- Cost
- Trust: is the tool handling your credentials securely?
- Cloud uploads or third-party APIs

I want to give you an alternative: *An offline, transparent method that gives you full control.*

And more than that, I want to share this knowledge as widely as I can. I'm lucky to have a network of brilliant, curious people, and I believe that if even a handful of us apply this... we make the world just a little safer.

Let's dive in.

My lab setup: Simulating the problem

To simulate a realistic risk scenario, I created a test Active Directory environment with 100 users. I then used PowerShell to assign each of them a password randomly chosen from a list of the top 20 most commonly used passwords, the kind that are technically complex enough to pass AD's default policy, but still dangerously predictable.

The result? Every single one of them was accepted without issue.

[Get the demo script from my GitHub repo.](#)

That's exactly the problem. These passwords feel secure because they satisfy the policy, but many are already known and actively exploited.

How do we check passwords against breach data?

In the rest of this post, I'll walk you through how to:

1. Download the NTLM hash list from [HavelBeenPwned](#).
2. Back up your `ntds.dit` SYSTEM registry hive.
3. Extract password hashes using PowerShell and DSInternals.
4. Compare those hashes against the HIBP data.
5. Remove all traces securely.

Step 1: Download the HavelBeenPwned NTLM hashes

We'll be working with the [PwnedPasswordsDownloader](#), a .NET tool that lets you download Troy Hunt's curated list of leaked password hashes.

For our purposes, make sure to download the NTLM hash version:

```
# First, install the .NET 8 SDK (not just the ASP.NET Runtime)
https://dotnet.microsoft.com/en-us/download/dotnet/8.0
```

```
# Install the downloader
dotnet tool install --global haveibeenpwned-downloader
```

```
# If you get a NuGet source error:
dotnet nuget add source "https://api.nuget.org/v3/index.json" --name "nuget.org"
```

```
# Download the NTLM hashes to a text file (this will take a while!)
haveibeenpwned-downloader.exe -n pwnedpasswords_ntlm
```

This will give you a massive file called `pwnedpasswords_ntlm.txt`, containing millions of NTLM hashes collected from real-world data breaches.

Note! At the time of writing this blog the download is already 45Gigs in size, make sure you have sufficient space available!

Why NTLM specifically?

Active Directory stores user passwords as **NTLM hashes** by default, yes even in modern environments. While Kerberos is used for authentication, the actual password data (as stored in `ntds.dit`) is hashed using the legacy NTLM algorithm.

That means:

- When we extract hashes from AD using DSInternals (*later in this blog*), we get NTLM hashes
- To compare them accurately, we need the same format from HavelBeenPwned

TL;DR: We're speaking the same "hash language" as AD — and that language is NTLM.

By using NTLM-based breach data, we can do a direct one-to-one comparison without needing to reverse or rehash anything.

Step 2: Back up your Active Directory database

Before we can analyze any passwords, we need to extract the relevant data from Active Directory:

- The `ntds.dit` database (which contains password hashes)
- The SYSTEM registry hive (which contains the boot key)

But let's get one thing very clear before you begin:

Warning: This is a highly sensitive operation!

You'll be accessing core components of your Domain Controller, files that were never meant to leave the system. Mishandling them can cause serious security risks or system instability.

Always use an isolated lab environment or a snapshot to gain familiarity with the process before doing this in production.

Requirements:

- You must be a **Domain Admin**
- You need to start an **elevated (Run as Administrator)** Command Prompt
- The target system must be a Domain Controller (virtual or physical)

We'll use the `ntdsutil` tool, which is included in Windows Server by default. Specifically, we'll use the `ifm` (Install From Media) option.

What does `ifm` do?

The `ifm` command creates a full snapshot of the Active Directory database and related registry files. Originally, it was designed to help set up new Domain Controllers from media. But in this context, it gives us the cleanest, most stable way to extract what we need.

Here's the process:

```
ntdsutil
ntdsutil: activate instance ntds
ntdsutil: ifm
ifm: create full D:\temp
ifm: quit
ntdsutil: quit
```

```
PS C:\Users\Superuser> ntdsutil
C:\Windows\system32\ntdsutil.exe: activate instance ntds
Active instance set to "ntds".
C:\Windows\system32\ntdsutil.exe: ifm
ifm: create full c:\temp
Creating snapshot...
Snapshot set {bf2cff5e-16b1-410a-8ab0-b7ccb2cc1716} generated successfully.
Snapshot {2302b994-f68d-41ab-903c-f1210b2cdd84} mounted as C:\$SNAP_202504101136_VOLUMEC$\
Snapshot {2302b994-f68d-41ab-903c-f1210b2cdd84} is already mounted.
Initiating DEFRAGMENTATION mode...
    Source Database: C:\$SNAP_202504101136_VOLUMEC$\Windows\NTDS\ntds.dit
    Target Database: c:\temp\Active Directory\ntds.dit

          Defragmentation   Status (complete)

0      10      20      30      40      50      60      70      80      90      100
|----|----|----|----|----|----|----|----|----|----|
.....

Copying registry files...
Copying c:\temp\registry\SYSTEM
Copying c:\temp\registry\SECURITY
Snapshot {2302b994-f68d-41ab-903c-f1210b2cdd84} unmounted.
IFM media created successfully in c:\temp
ifm: quit
C:\Windows\system32\ntdsutil.exe: quit
PS C:\Users\Superuser>
```

The process of dumping the `ntds.dit`

This creates a set of files under:

```
D:\temp\Active Directory\
D:\temp\registry\
```

What you now have:

- `ntds.dit`: The AD database with password hashes.
- `SYSTEM`: The registry hive containing the boot key.

Both of these will be needed in the next step to decrypt and extract NTLM hashes.

Pro tip: Once created, copy these files to a secure analysis machine and remove them from the DC immediately.

Step 3: Extract password hashes using DSInternals

Now that we have both the ntds.dit database and the matching SYSTEM registry hive from the Domain Controller, we can extract the password hashes for analysis.

We'll use the awesome DSInternals PowerShell module by Michael Grafnetter, a trusted tool for forensic analysis and low-level Active Directory operations. Cool side note that Michael is also a huge StarWars fan like myself, must be in the name!

Install DSInternals (if you haven't already):

```
Install-Module -Name DSInternals
```

If prompted to install from an untrusted repository, type Y to confirm (or use -Force to skip the prompt in test environments).

Step-by-step extraction process

Now let's extract the password hashes from the .dit file using the boot key from the SYSTEM hive.

1. Load the boot key from the SYSTEM registry hive:

```
$Key = Get-BootKey -SystemHiveFilePath "D:\Temp\registry\SYSTEM"
```

This decrypts the SYSKEY stored in the SYSTEM registry, the key we need to unlock encrypted content inside ntds.dit.

Why is this important?

The ntds.dit database is **not portable** between machines. It is cryptographically bound to the specific Domain Controller it was created on. This is by design: Microsoft protects the sensitive contents of Active Directory (like NTLM hashes and Kerberos secrets) using a combination of encryption layers.

One of those layers is the boot key (SYSKEY), stored in the SYSTEM registry hive, which unlocks additional keys that are used internally by AD to encrypt password data. Without this specific boot key, the content of ntds.dit cannot be decrypted.

That means if you copy the .dit file from DC01 and try to read it on another system without the matching SYSTEM hive from DC01, it simply won't work. This ensures that password data remains secure, even if the database is exfiltrated.

2. Extract password hashes from the AD database:

```
Get-ADDBAccount -BootKey $Key `
  -DatabasePath D:\Temp\Active Directory\ntds.dit" `
  -All `
  -ExportFormat HashcatNT |
  Out-File D:\Temp\Hashdump.txt
```

This command does the following:

- Decrypts the `ntds.dit` database using the boot key
- Extracts all available NTLM password hashes from all user objects
- Outputs them in *Hashcat-compatible NTLM format* to a plain text file

Each line in the resulting `Hashdump.txt` will look something like (*username:hash*):

```
Administrator:217e50203a5aba59cefa863c724bf61b
Guest:
Superuser:217e50203a5aba59cefa863c724bf61b
krbtgt|:25e4da8ef02c445535a0ed59bc4de2d6
lab381279:5296223f66fb3dfd02cfe5d27e261868
lab849059:e4b94dfa211824b37b3f32088898d4fd
lab919705:5296223f66fb3dfd02cfe5d27e261868
lab491565:01d460d6aa5cd5dfa8edf093f4947e38
lab257198:12a75a358ba2b65d03890add7550d1ac
lab984532:520126a03f5d5a8d836f1c4f34ede7ce
lab850230:ec9f6f5729d117cdd8db67a104a501a3
lab352864:7facdc498ed1680c4fd1448319a8c04f
lab253548:12a75a358ba2b65d03890add7550d1ac
lab351047:12a75a358ba2b65d03890add7550d1ac
lab693524:5f5cadac80a83e69dd194a232d2d89c8
lab116483:e3f85bb089071e07b4a5662d644fa131
lab438252:f44140efb4542311a2184c37917c93d4
lab384334:01d460d6aa5cd5dfa8edf093f4947e38
lab713916:9307ee5abf7791f3424d9d5148b20177
lab972701:6b6a4edf8583b5aa760921ff211ee399
lab637889:5f5cadac80a83e69dd194a232d2d89c8
```

Overview of extracted credentials

Why Hashcat format? It's standardized, widely supported, and works seamlessly with matching tools and password auditing workflows. At this point, you now have:

- A list of all AD user accounts
- Their corresponding NTLM password hashes
- A secure local file (`Hashdump.txt`) ready for comparison

Step 4: Compare hashes using Match-ADHashes

Now that we've extracted all password hashes from Active Directory and downloaded the massive NTLM hash list from [HaveIBeenPwned](#), it's time to compare the two.

To do this, we'll use a PowerShell module called Match-ADHashes, developed by [DGG-IT](#). It's specifically designed to compare AD hash dumps to public breach data in an efficient, offline and transparent way, without sending anything externally.

Requirements:

- Hashdump.txt: the NTLM hashes exported from your AD environment
- pwnedpasswords_ntlm.txt: the HIBP NTLM hash list you downloaded earlier

Step-by-step instructions

1. Download and import the comparison module:

First, clone or download the script from GitHub:

```
git clone https://github.com/DGG-IT/Match-ADHashes.git
cd Match-ADHashes
Import-Module .\Match-ADHashes.ps1 -Force
```

If you're not using Git, just download the .ps1 file manually and place it in your working directory.

2. Run the hash comparison:

```
$list = Match-ADHashes `
    -ADNTHashes "D:\Temp\Hashdump.txt" `
    -HashDictionary "D:\data\pwnedpasswords_ntlm.txt"
```

What this does:

- It reads each NTLM hash from your AD dump
- It checks for matches in the breach data
- If a match is found, it records:
 - The matched hash
 - The number of times it appeared in breaches (Frequency)
 - The associated AD user(s)

Note! This will take a long time, usually an hour or more depending on the number of users and your available hardware.

3. Export the results to a CSV:

```
$list | Select Hash, Frequency, @{Name='User'; Expression = {[string]::Join(";",
$_.User)}} |
Export-Csv -Path "C:\Temp\Matched-Hashes.csv" -Delimiter ';' -NoTypeInfo
```

This will give you a clean report with every matched user and how common their password is in public breaches. A high frequency value (e.g. 2,345,678) means that password has

been reused and leaked millions of times.

Example output:

User	Frequency	Hash
{lab491565, lab384334, lab149216, lab549150...}	205	01D460D6AA5CD5DFA8EDF093F4947E38
{lab257198, lab253548, lab351047, lab203673...}	80	12A75A358BA2B65D03890ADD7550D1AC
{Administrator, Superuser}	76603	217E50203A5ABA59CEFA863C724BF61B

If you see results like these, those users are walking around with time bombs for passwords, and attackers are already holding the match. Change those passwords immediately.

User	Frequency	Hash
{lab491565, lab384334, lab149216, lab549150...}	205	01D460D6AA5CD5DFA8EDF093F4947E38
{lab257198, lab253548, lab351047, lab203673...}	80	12A75A358BA2B65D03890ADD7550D1AC
{Administrator, Superuser}	76603	217E50203A5ABA59CEFA863C724BF61B
{lab885044, lab288597}	85	37785371438A460949C4D6AD23645AA9
{lab492774, lab826667, lab685477, lab779641}	61	485934D8D427E7FA7C190EEC81E9208D
{lab984532, lab106809}	205994	520126A03F5D5A8D836F1C4F34EDE7CE
{lab381279, lab919705, lab494481, lab856717}	5867	5296223F66FB3DFD02CFE5D27E261868
{lab718878, lab741277, lab601012, lab798770...}	1271	5A442B8EC922E066CBDD5E793A565B73
{lab693524, lab637889, lab610235, lab175261...}	12144	5F5CADAC80A83E69DD19A232D2D89C8
{lab972701, lab215595, lab818406, lab908061}	1487	6B6A4EDF8583B5AA760921FF211EE399
{lab352864, lab953772, lab658831}	320782	7FACDC498ED1680C4FD1448319A8C04F
{lab713916, lab680743, lab521577, lab626810...}	123487	9307EE5ABF7791F3424D9D5148B20177
{lab430007, lab514269, lab816572, lab308263...}	35	ACAD63DE336CB88DF37C0DBDEF1504F9
{lab352117, lab175433, lab696063, lab165864}	30441	DD5ADD482832198C420757E0929174EE
{lab116483, lab764699, lab266981, lab109249}	23	E3F85BB089071E07B4A5662D644FA131
{lab849059, lab862985, lab342740, lab200344}	68	E4B94DFA211824837B3F32088898D4FD
{lab850230, lab387936, lab923810, lab181212...}	29	EC9F6F5729D117CDD8DB67A104A501A3
{lab438252, lab447465, lab702243, lab656863...}	7	F44140EFB4542311A2184C37917C93D4
{lab227458, lab437998, lab276428, lab129251...}	2280	F4CDD6C7BD7A1510D3CF64D100D6EF08

All accounts using leaked passwords

Bonus insight: What if someone gets the hashes?

It's tempting to think that "*a hash isn't a password*", and that storing hashes isn't as *bad* as storing cleartext credentials. But unfortunately, that's not true.

If someone gets their hands on the NTLM hashes you extracted, even without the original passwords, they might still be able to do lateral movement with tools like [MimiKatz](#) or reverse them using public hash databases like [hashes.com](#).

How? Sites like hashes.com maintain massive lookup tables of pre-cracked password hashes, collected from public breaches and cracking sessions. If your hash appears in their database, anyone can look up the *original* password in plain text... in seconds.

A match on hashes.com doesn't just prove the password is weak — it gives attackers the **exact password** to try against other systems.

Deleting the hash files after use isn't just good practice, it protects you from situations where your own internal audit data becomes the next attack vector. Hence the last chapter for today.

Step 5: Secure cleanup after testing

Now that your analysis is complete, it's time for what might be the most important step of all: *cleaning up your sensitive files*.

During this process, you've worked with:

- A full copy of your `ntds.dit` Active Directory database
- The SYSTEM registry hive containing your domain controller's boot key
- A list of extracted NTLM password hashes for all users in your environment
- A massive file with billions of known compromised password hashes

All of these files, especially the extracted hash dump, are **extremely sensitive**. If they end up in the wrong hands, they can be used to crack passwords offline and gain unauthorized access to your systems.

Golden Rule: If you're not actively analyzing, you should not be storing.

These files should **never** be:

- Left behind in temporary directories
- Synced to cloud storage like OneDrive or Google Drive
- Emailed, compressed, or shared outside secured environments

How to securely remove the files

Use the following steps to delete the sensitive files and overwrite free space on the drive. This helps prevent file recovery by forensic tools.

1. Remove the extracted hash dump

```
Remove-Item "D:\Temp\Hashdump.txt" -Force
```

2. Remove the downloaded breach data

```
Remove-Item "D:\data\pwnedpasswords_ntlm.txt" -Force
```

3. Overwrite deleted data with zeroes (Windows built-in tool)

```
cipher /w:D:\Temp
```

```
cipher /w:D:\data
```

cipher /w: will write over free space on the drive with zeroes. It won't delete individual files, but it makes deleted data much harder to recover.

Recommendations: Now what?

Now that you've identified weak or compromised passwords in your environment, the next step is acting on that insight. This is where you move from *visibility* to *improvement*.

Here are my key recommendations:

Reset any exposed passwords immediately If a password was found in the breach data — no matter how “complex” it looks — it should be considered compromised and reset as soon as possible. Don't assume users will do it themselves. Enforce it.

Repeat this check every 3 to 6 months This is not a one-time thing. Users tend to fall back into bad habits over time, especially when forced to change passwords regularly.

Implement Azure AD Password Protection

Microsoft offers [Password Protection for Active Directory](#), both in the cloud and on-prem (with a proxy and agent setup). It blocks known weak passwords *at the time of creation*, including custom banned word lists.

Don't rely on password complexity rules alone

“Complex” doesn't mean secure. Users will still find a way to create predictable passwords like Summer2024! or Password1!. Focus on **length, uniqueness, and non-reuse** instead.

Use MFA everywhere, and aim for passwordless

Wherever possible, strengthen authentication with Multi-Factor Authentication. But long term, the goal should be eliminating passwords altogether.

Whether it's via Windows Hello for Business, FIDO2 keys, or other platform authenticators, ***passwordless is the only real way forward.***

| *Good security is never static, it's a cycle of awareness, action, and adaptation.*

You've taken the first step. Now keep going. Because a password that's already public... is no longer a secret.

As always, if you found this information useful, have a question or remark, please let me know. Until next time!